Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Literature Review
5. Objectives
6. Technologies Used
7. System Architecture
8. Application Design and Flow
9. CORS: Conceptual Overview
10. Flask-Based Implementation
11. Security Implications of CORS
12. Real-World Use Cases
13. Code Walkthrough and Analysis
14. Database Schema
15. Admin Panel and API Log Monitoring
16. Public and Private Routes
17. Attack Demonstrations
18. Test Cases and Result Analysis
19. Deployment Options and Cloud Hosting
20. Future Enhancements
21. Conclusion
22. References
23. Appendix

1. Abstract

In the evolving landscape of web development, security and accessibility are paramount. Cross-Origin Resource Sharing (CORS) is a security mechanism implemented by web browsers to prevent unauthorized data access across different origins. As modern applications grow increasingly dependent on distributed systems and API-based architectures, the need for secure cross-origin communication becomes essential.

This project investigates the importance of CORS in web development, its implementation in Python using Flask, and the consequences of misconfiguration. It includes a practical demonstration involving both secure and insecure CORS-enabled routes. Emphasis is placed on real-world attack simulations, secure deployment practices, and common vulnerabilities such as overly permissive CORS settings. The final output is a research-driven, Flask-based application with a secure admin interface, public API endpoints, and robust CORS policies tailored for educational and commercial use.

2. Introduction

The development of modern web applications often requires interaction between services hosted on different domains. While this architectural choice increases scalability and flexibility, it introduces new security challenges. Browsers enforce the same-origin policy, which restricts how documents or scripts loaded from one origin can interact with resources from another. To enable legitimate cross-origin communication, CORS was introduced as a standard.

This report explores how CORS is a necessary tool in enabling secure resource sharing between different web domains. We implement and analyse a full-stack web application using Python (Flask) that allows interaction between separate frontend and backend servers under a CORS policy.

We also study how misconfigurations in CORS settings can introduce vulnerabilities like credential leaks, cross-site attacks, and unauthorized access. The report includes implementation strategies, backend API development, frontend integration, and monitoring of CORS-related requests and logs in an admin dashboard.

3. Problem Statement

Despite the critical importance of CORS in web application security, many developers and researchers lack in-depth knowledge of its configuration and impact. Common mistakes such as wildcard `Access-Control-Allow-Origin: *`, incorrect handling of credentials, or trusting dynamic origins can lead to severe security breaches.

This project aims to bridge the gap between theoretical understanding and practical implementation of CORS. It addresses the problem of safely enabling cross-origin requests while maintaining control over access, demonstrating both best practices and insecure setups.

4. Literature Review

A thorough study of available documentation, research papers, and industry best practices was conducted to understand:

- The evolution of the Same-Origin Policy (SOP) and why it's not sufficient.
- CORS specification by W3C and its adoption in browsers.
- Case studies on CORS misconfigurations (e.g., Starbucks, Alibaba, and Uber security flaws).
- Comparisons between JSONP, CORS, and alternatives.
- Industry recommendations by OWASP and Mozilla Developer Network.

5. Objectives

Primary Objectives:

- To implement a secure, CORS-enabled web application using Python Flask.
- To simulate CORS-enabled and restricted scenarios.
- To demonstrate real-world attacks and misconfiguration risks.
- To create a monitoring dashboard to audit cross-origin traffic.

Secondary Objectives:

- To understand the implications of exposing private APIs.
- To evaluate the effectiveness of different CORS policies.
- To offer a reusable project template for educational and research purposes.

6. Technologies

Technology	Role
Python	Backend development
Flask	Web application framework
Flask-CORS	CORS configuration
SQLite	Lightweight database for storing logs and data
HTML/CSS/JS	Frontend
Bootstrap	UI components and styling
Postman	API testing
Git/GitHub	Version control and project sharing

7. System Architecture

A diagram illustrating communication between:

- Frontend hosted on `localhost:3000`
- Backend API at `localhost:5000`
- Database layer for storing inquiries, feedback, and API logs
- Admin dashboard

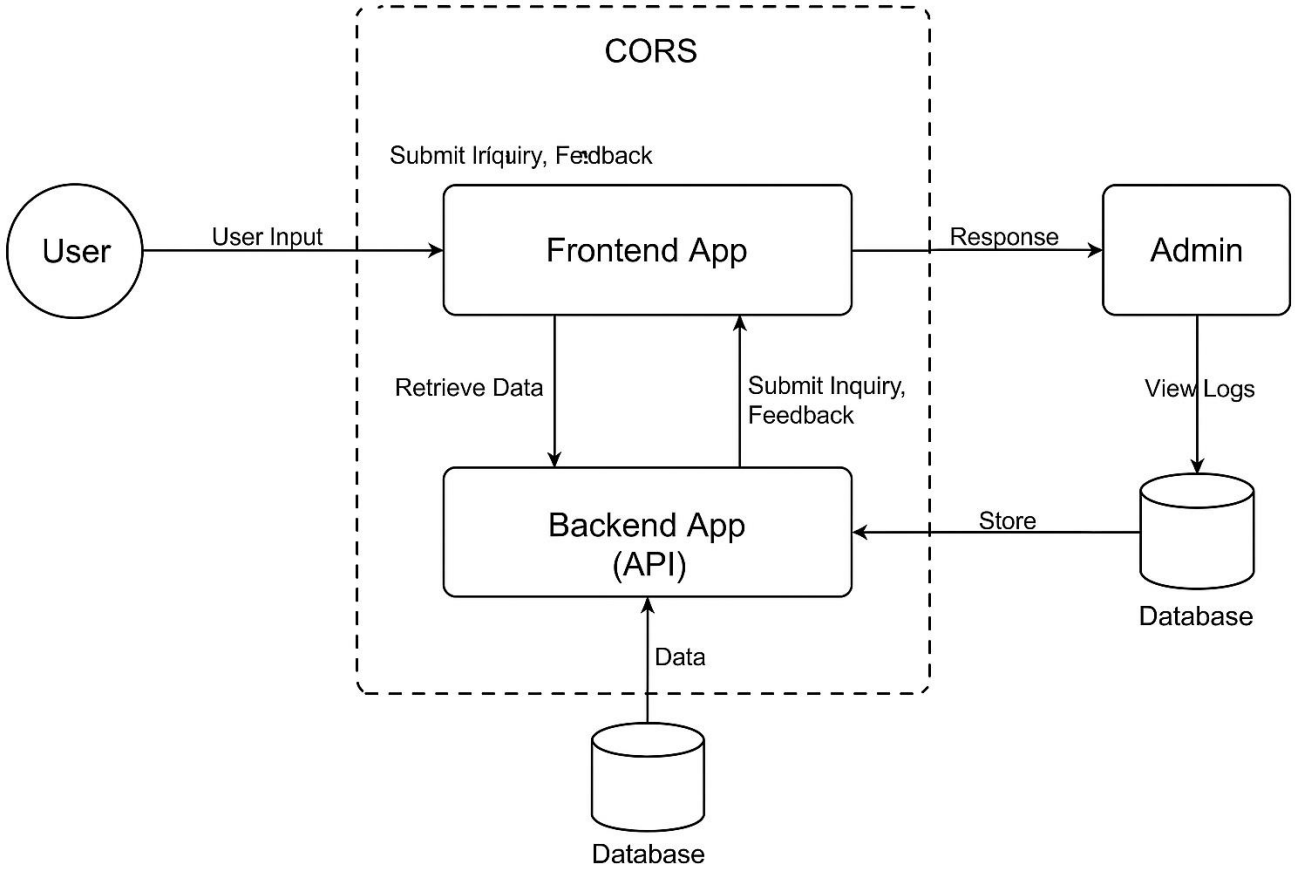
- Request/response flow
- CORS preflight checks (OPTIONS requests)
- Browser enforcement of policies

8. Application Design and Flow

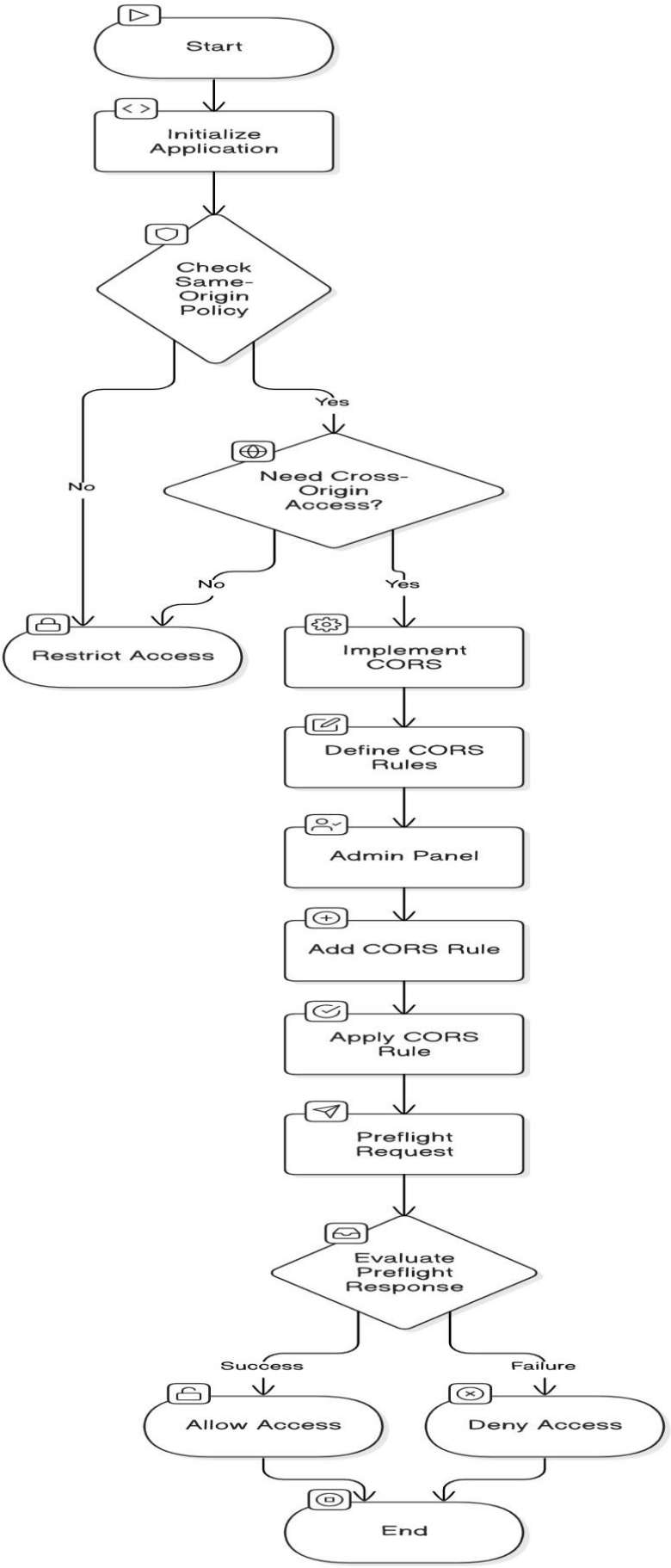
Include flowcharts or sequence diagrams showing:

- Admin login and session handling
- Public route access without credentials
- Secure route access requiring CORS and token
- Inquiry and feedback form submission (write operations)

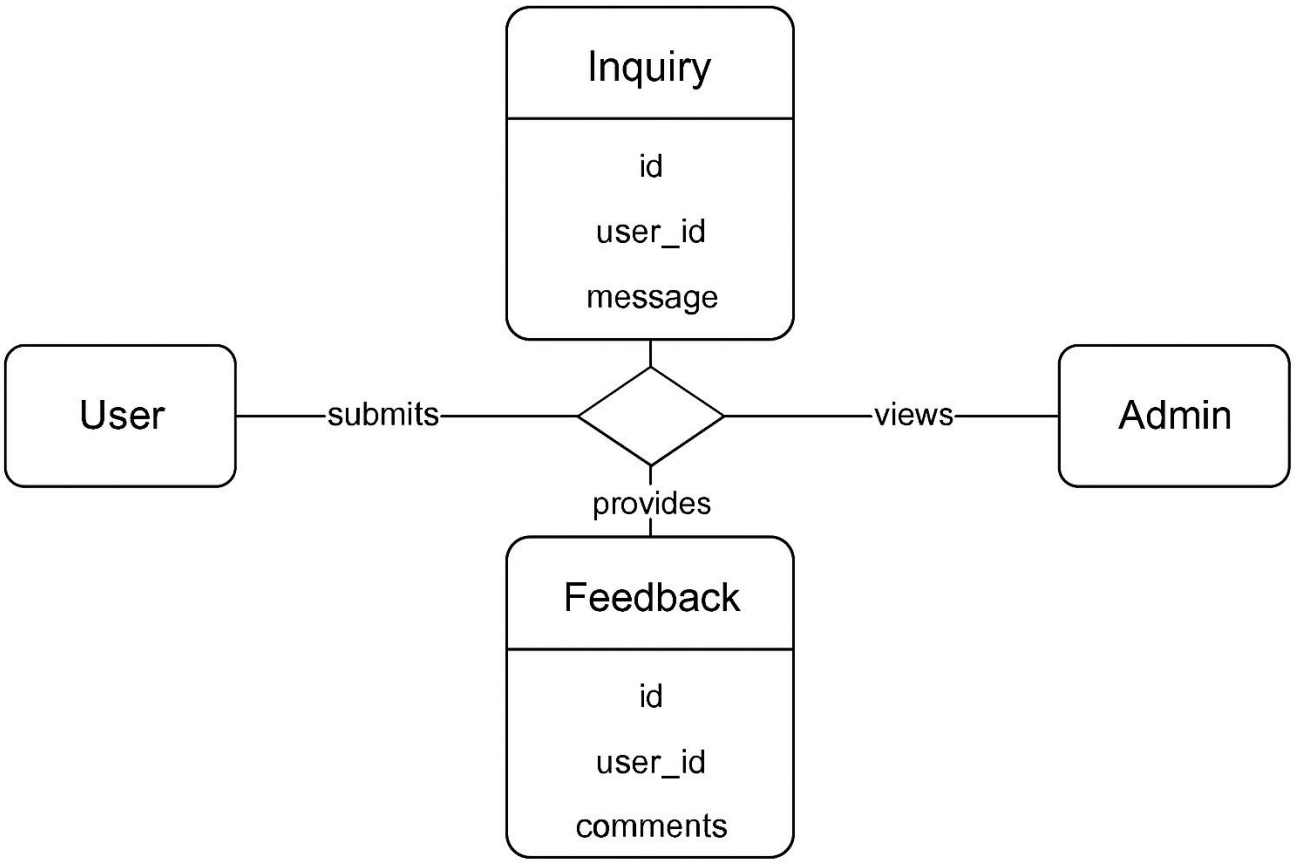
Application Flow:



CORS Implementation in Web Application



ER Diagram


Inquiry
id
user_id
message
User
submits
views
Admin
provides
Feedback
id
user_id
comments

9. Conceptual Overview of CORS

- Definition of CORS and the Same-Origin Policy

Here's a clear and detailed explanation of **CORS** and the **Same-Origin Policy**:

Same-Origin Policy (SOP)

Definition: The **Same-Origin Policy** is a critical security concept implemented by web browsers that restricts how documents or scripts loaded from one origin can interact with resources from another origin.

Origin is defined as the combination of:

- **Protocol (scheme)** – e.g., `http` or `https`
- **Host (domain)** – e.g., `example.com`
- **Port number** – e.g., `:80`, `:443`

Two URLs are considered to have the same origin **only if all three components are identical**.

Example:

URL	Same Origin as `http://example.com`?
`http://example.com`	☐ Yes
`https://example.com`	☐ No (different protocol)
`http://sub.example.com`	☐ No (different subdomain)
`http://example.com:8080`	☐ No (different port)

Purpose: SOP is designed to prevent **malicious scripts** on one site from accessing sensitive data on another through the browser.

Cross-Origin Resource Sharing (CORS)

Definition: CORS (Cross-Origin Resource Sharing) is a **browser-based security mechanism** that allows controlled access to resources located outside of a given origin (i.e., cross-origin requests).

It is a **relaxation of the Same-Origin Policy**—it defines a secure way for web applications to request resources from another domain/server.

How it works: When a cross-origin HTTP request is made, the browser adds a special header (`Origin`) to the request. The server responds with **CORS headers** like:

- `Access-Control-Allow-Origin`
- `Access-Control-Allow-Methods`
- `Access-Control-Allow-Headers`

If the response headers meet the browser's CORS policy, the response is made available to the requesting web application.

Example: If your frontend (at `http://frontend.com`) wants to fetch data from an API at `http://api.backend.com`, the backend must include:

```
Access-Control-Allow-Origin: http://frontend.com
```

in its response for the browser to allow the response to be accessed by the frontend.

Summary

Concept	Purpose	Enforced By
Same-Origin Policy	Prevent unauthorized access across origins	Browser
CORS	Allow secure access across origins (if permitted)	Browser + Server Configuration

- How browsers enforce CORS

How Browsers Enforce CORS

CORS enforcement is handled **entirely by browsers** to protect users from cross-origin attacks. The server doesn't enforce CORS—it simply responds with the appropriate headers. It's the browser that checks these headers and decides whether or not to allow the request/response.

Step-by-Step Process

1. Browser Sends an HTTP Request with Origin Header

When a script on a webpage (e.g., using `fetch()` or `XMLHttpRequest`) tries to access a resource on a **different origin**, the browser automatically includes an `Origin` header:

```
Origin: http://example.com
```

2. Server Responds with CORS Headers

The server receiving the request decides whether or not to allow it by sending appropriate CORS headers, like:

```
Access-Control-Allow-Origin: http://example.com
```

3. Browser Checks the Response Headers

- If the `Access-Control-Allow-Origin` **matches the request's origin** (or is a wildcard `*` in certain cases), the browser allows the script to access the response.
- If not, the browser **blocks access** to the response data and shows a **CORS error** in the console.

4. Preflight Request (for Non-Simple Requests)

For some requests (e.g., with custom headers, PUT/DELETE methods), the browser sends a **preflight request** using the `OPTIONS` method before the actual request.

Preflight includes:

```
OPTIONS /api/data HTTP/1.1
Origin: http://example.com
Access-Control-Request-Method: POST
Access-Control-Request-Headers: X-Custom-Header
```

Server must respond with:

```
Access-Control-Allow-Origin: http://example.com
Access-Control-Allow-Methods: POST
Access-Control-Allow-Headers: X-Custom-Header
```

Only if the response is valid does the browser proceed with the actual request.

What Happens If CORS Fails?

- The browser **blocks access** to the response (even if the request itself succeeds).
- A **CORS error** appears in the console like:

  "Access to fetch at 'http://api.com/data' from origin 'http://yourdomain.com' has been blocked by CORS policy."

- **Important:** This is a browser-level protection. If you bypass the browser (e.g., using cURL, Postman, or server-side code), CORS is **not enforced**.

- Structure of a CORS response:
  - `Access-Control-Allow-Origin`
  - `Access-Control-Allow-Headers`
  - `Access-Control-Allow-Methods`
  - Preflight requests (OPTIONS)

- Types of requests:
  - Simple requests
  - Preflighted requests
  - Credentialed requests

10. Flask-Based Implementation

- Flask setup
- Flask-CORS installation and usage
- Applying CORS globally vs. route-specific
- Secure headers with credentials
- Handling preflight (OPTIONS) requests

Front end application – Using HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Web CORS Demo - Home</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css" rel="stylesheet">
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <a class="navbar-brand ms-3" href="index.html">CORS Web Demo</a>
    <div class="collapse navbar-collapse">
      <ul class="navbar-nav ms-auto me-3">
        <li class="nav-item"><a class="nav-link"
href="about.html">About</a></li>
        <li class="nav-item"><a class="nav-link"
href="inquiry.html">Inquiry</a></li>
        <li class="nav-item"><a class="nav-link"
href="feedback.html">Feedback</a></li>
        <li class="nav-item"><a class="nav-link"
href="services.html">Services</a></li>
        <li class="nav-item"><a class="nav-link" href="secure-
data.html">Secure Data</a></li>
        <li class="nav-item"><a class="nav-link" href="cors-
demo.html">CORS Demo</a></li>
        <li class="nav-item"><a class="nav-link"
href="logs.html">Logs</a></li>
        <li class="nav-item"><a class="nav-link"
href="contact.html">Contact</a></li>
      </ul>
    </div>
  </nav>

  <div class="container mt-4">
    <h1 class="text-center">Welcome to the CORS Web Application
Demo</h1>
    <p class="lead text-center">This app demonstrates how Cross-Origin
Resource Sharing (CORS) works in web applications using Flask (Python)
and HTML + JavaScript.</p>

    <div class="row mt-5">
      <div class="col-md-6">
        <h4>⚙ Features</h4>
        <ul>
```

```
<li>AJAX-based Inquiry and Feedback forms</li>
          <li>Cross-origin requests with CORS enabled</li>
          <li>Secure vs Public APIs demo</li>
          <li>Real-time admin log viewer</li>
        </ul>
      </div>
      <div class="col-md-6">
        <h4>□ Try It Yourself</h4>
        <p>Use the navigation bar to test different pages:</p>
        <ol>
          <li>Submit inquiries</li>
          <li>Send feedback</li>
          <li>Check CORS policy behavior</li>
          <li>View secure/private route handling</li>
        </ol>
      </div>
    </div>
  </div>

  <footer class="bg-dark text-white text-center mt-5 p-3">
    &copy; 2025 CORS Web Demo. All rights reserved.
  </footer>
</body>
<script src="app.js"></script>

</html>
```

Secure-data.html file

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Secure Data - CORS Demo</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.m
in.css" rel="stylesheet">
  <script>
    async function fetchSecureData() {
      const token = document.getElementById("tokenInput").value ||
"demo-token";

      try {
        const response = await fetch("http://127.0.0.1:5000/secure-
data", {
          method: 'GET',
          headers: {
            "Authorization": "Bearer " + token
          }
        });

        if (!response.ok) {
          throw new Error("Error: " + response.status + " " +
response.statusText);
        }

        const data = await response.json();
        document.getElementById("dataDisplay").textContent =
JSON.stringify(data, null, 2);
      } catch (err) {
        document.getElementById("dataDisplay").textContent =
err.message;
      }
    }
  </script>
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <a class="navbar-brand ms-3" href="index.html">CORS Web Demo</a>
    </nav>

    <div class="container mt-4">
      <h2>Secure Data via CORS</h2>
      <p>Enter your Authorization token to fetch protected data.</p>
      <div class="mb-3">
```

```
<input type="text" id="tokenInput" class="form-control"
placeholder="Enter token (e.g., demo-token)">
        <button onclick="fetchSecureData()" class="btn btn-primary mt-
2">Fetch Data</button>
      </div>
      <pre id="dataDisplay" class="bg-light p-3 border">Waiting for
token...</pre>
    </div>

    <script>
      async function fetchSecureData() {
        const token = document.getElementById("tokenInput").value;
        try {
          const response = await fetch("http://127.0.0.1:5000/secure-
data", {
            headers: {
              "Authorization": "Bearer " + token
            }
          });

          if (!response.ok) {
            document.getElementById("dataDisplay").textContent =
"Unauthorized or Error: " + response.status;
            return;
          }

          const data = await response.json();
          document.getElementById("dataDisplay").textContent =
JSON.stringify(data, null, 2);
        } catch (err) {
          document.getElementById("dataDisplay").textContent = "Error:
" + err.message;
        }
      }
    </script>
  </body>

</html>
```

Backend application – Python app

```
from flask import Flask
from flask_cors import CORS
from config import Config
from db import db
from routes import public_routes, secure_routes, log_routes,
admin_routes

app = Flask(__name__)
app.config.from_object(Config)

db.init_app(app)
# Enable CORS for all routes and allow only your frontend origin
CORS(app, resources={r"/*": {"origins": "http://127.0.0.1:5500"}})

app.register_blueprint(public_routes.bp)
app.register_blueprint(secure_routes.bp)
app.register_blueprint(log_routes.bp)
app.register_blueprint(admin_routes.bp)

if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

Database connection:

Db.py file

```
import os

class Config:
    SECRET_KEY = 'super-secret-key'
    SQLALCHEMY_DATABASE_URI = 'sqlite:///site1.db'
    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

Secure_routes.py file

```
from flask import Blueprint, jsonify, request
from db import Log, db

bp = Blueprint('secure', __name__)

# @bp.route('/secure-data', methods=['GET'])
# def secure_data():
#     origin = request.headers.get('Origin', 'Unknown')
#     db.session.add(Log(origin=origin, endpoint='/secure-data',
method='GET', status=200))
#     db.session.commit()
#     return jsonify({'secure_info': 'This is protected data'}), 200

@bp.route('/secure-data', methods=['GET'])
def get_secure_data():
    auth_header = request.headers.get('Authorization', '')
    if auth_header != "Bearer demo-token":
        return jsonify({"error": "Unauthorized"}), 401

    return jsonify({"data": "This is protected data from Flask
backend"})
```

10. Security Implications

CORS is powerful, but **misconfigurations can lead to serious security vulnerabilities**, especially when sensitive data or user credentials are involved.

Common Misconfigurations

1. Wildcard Origin with Credentials

Using:

```
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
```

is **invalid** and **dangerous**.

- The wildcard (`*`) means **any origin** can access the response.
- `Allow-Credentials: true` tells the browser to include **cookies, tokens, or HTTP authentication**.
- But browsers **block this combination**, and **ignore the credentials**, leading to undefined and insecure behavior.

Fix: Specify the exact trusted origin when credentials are involved:

```
Access-Control-Allow-Origin: https://trusted-site.com
Access-Control-Allow-Credentials: true
```

2. Dynamically Reflecting Origin Without Validation

Some servers reflect the incoming `Origin` header like this:

```
Access-Control-Allow-Origin: <value from request Origin header>
Access-Control-Allow-Credentials: true
```

Without checking **if the origin is trusted**, this creates a **severe vulnerability**.

Risk: Any malicious site can send a request, have their origin reflected, and steal sensitive data.

Fix: Validate the origin against a **whitelist** of trusted domains before reflecting it in the response.

CORS-Related Attack Scenarios

1. Data Theft via Cross-Site API Access

An attacker creates a malicious website and tricks a logged-in user into visiting it. If the backend CORS policy is weak, the attacker's site can:

- Send a cross-origin request to the API
- Receive a sensitive response (e.g., user profile, messages, financial data)
- Extract and misuse the data

2. Unauthorized API Access

If CORS is too permissive, an attacker can interact with internal APIs or admin endpoints without proper authorization, using a malicious frontend.

Real-World Examples from Bug Bounty Reports

1. **Uber (2016)**
    - Bug bounty report found that Uber's API reflected origin headers and allowed credentials without validation.
    - Result: An attacker could steal private ride history of users.
    - □ Rewarded $7,500
2. **Facebook Subdomain (2019)**
    - Misconfigured CORS on a Facebook subdomain allowed attackers to bypass SOP and access cross-origin content.
    - □ Reported via HackerOne, high-severity.
3. **Tesla (2021)**
    - CORS misconfig allowed access to internal APIs from any origin.
    - Could be exploited for account takeover or data theft.
    - □ Significant reward paid.

Safe Patterns for Credentialed Requests

To safely allow **credentialed** cross-origin requests (with cookies, session tokens, or headers like `Authorization`):

1. **Use Specific Origins Only**
2. `Access-Control-Allow-Origin: https://trusted-client.com`
3. **Always Include `Vary: Origin` Header**
    - Ensures proper caching of responses for each origin
4. `Vary: Origin`
5. **Set `Access-Control-Allow-Credentials: true`**
    - Only when needed and when the origin is trustworthy
6. `Access-Control-Allow-Credentials: true`
7. **Use HTTPS Only**
    - Prevents man-in-the-middle attacks and hijacked CORS requests
8. **Maintain a Strict Whitelist**
    - Validate incoming `Origin` headers server-side against known origins
9. **Never Use Wildcard (`*`) with Credentials**

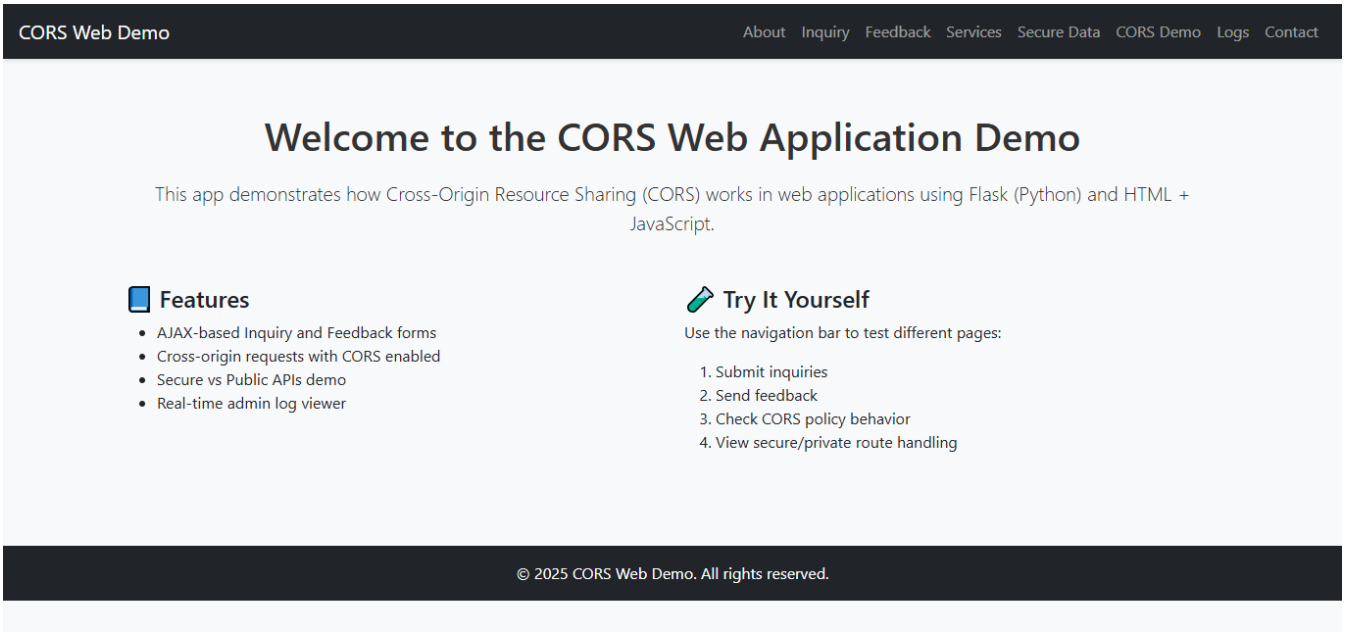
11. Real-World Use Cases

- SPAs using APIs (React, Angular apps with Flask backend)
- Third-party integrations: payment, login, notifications
- Microservices and API gateway communication

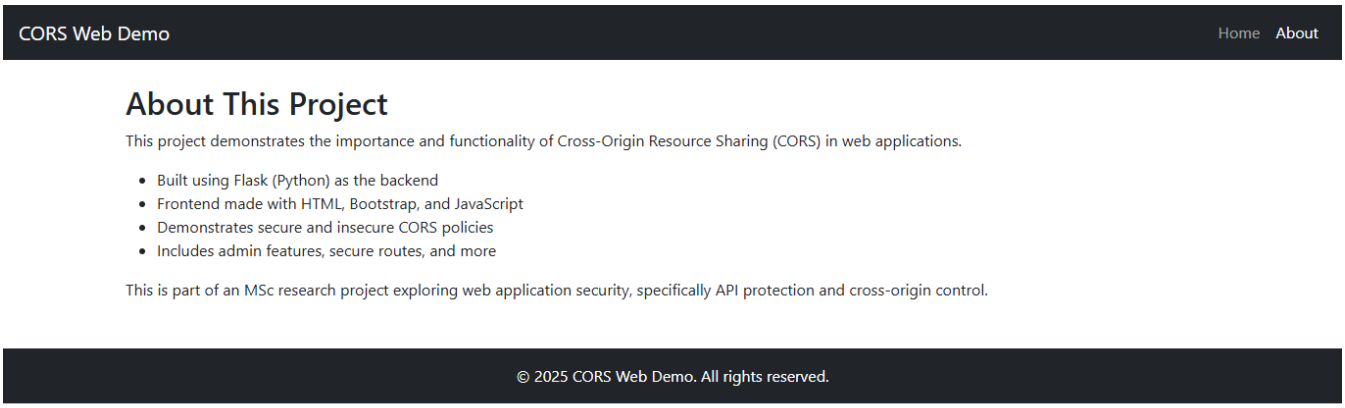
12. Code Walkthrough and Results

- `app.py` – entry point
- `routes/secure_routes.py` – protected APIs
- `routes/public_routes.py` – open APIs
- `admin/` – login, view logs
- `db.py` – SQLite functions

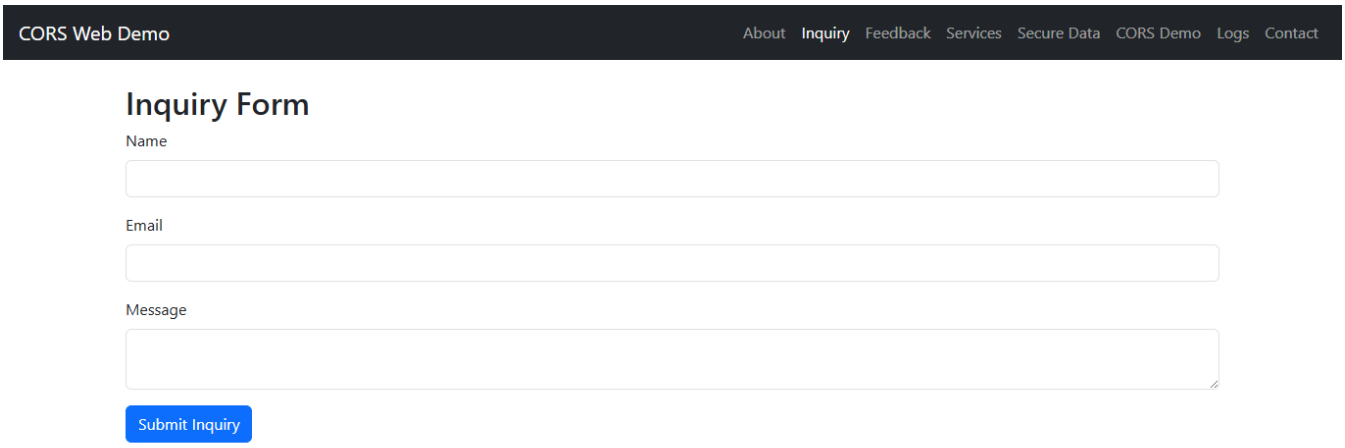
Home Page – Frontend Application



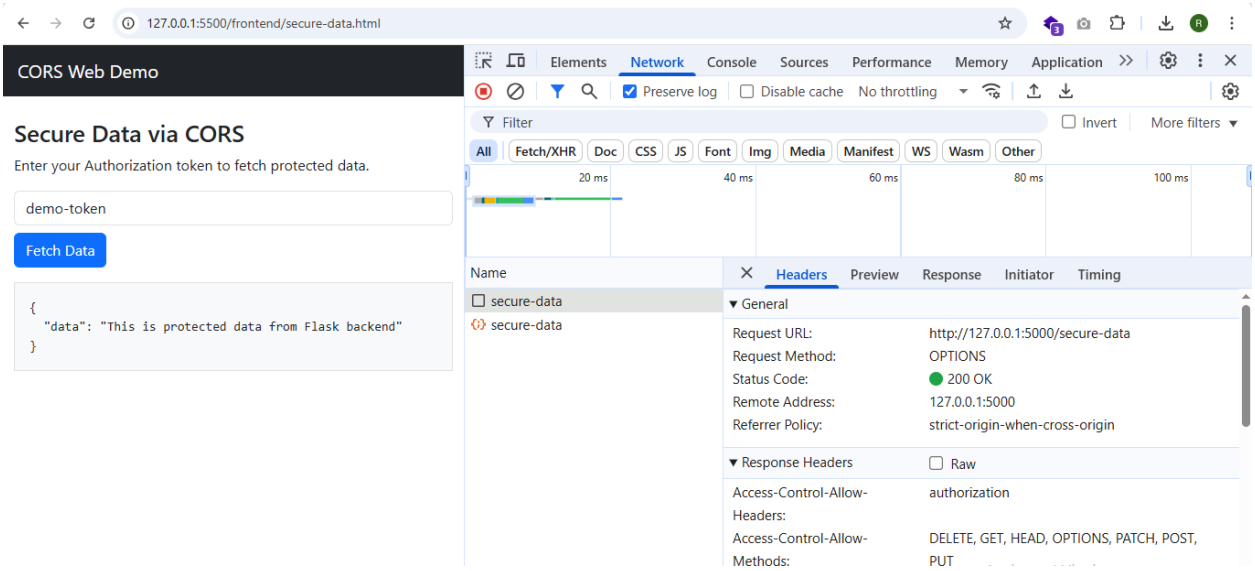
About Page:



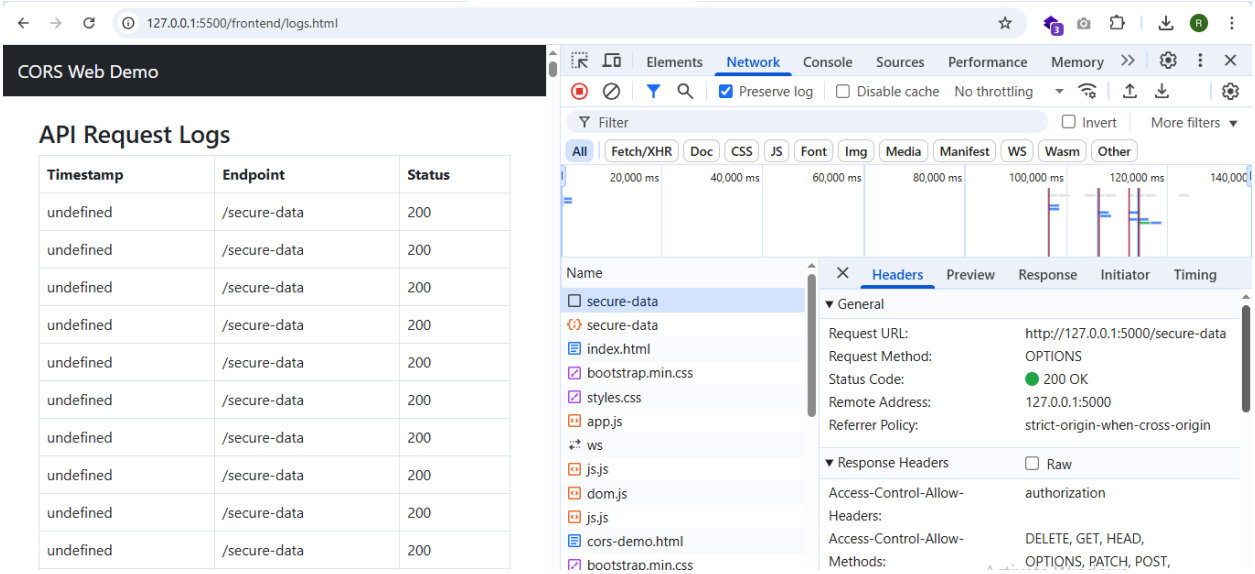
Inquiry Page



Secure Data route with CORS cross origin policy applied.



Logs:



Backend Application – Using Flask

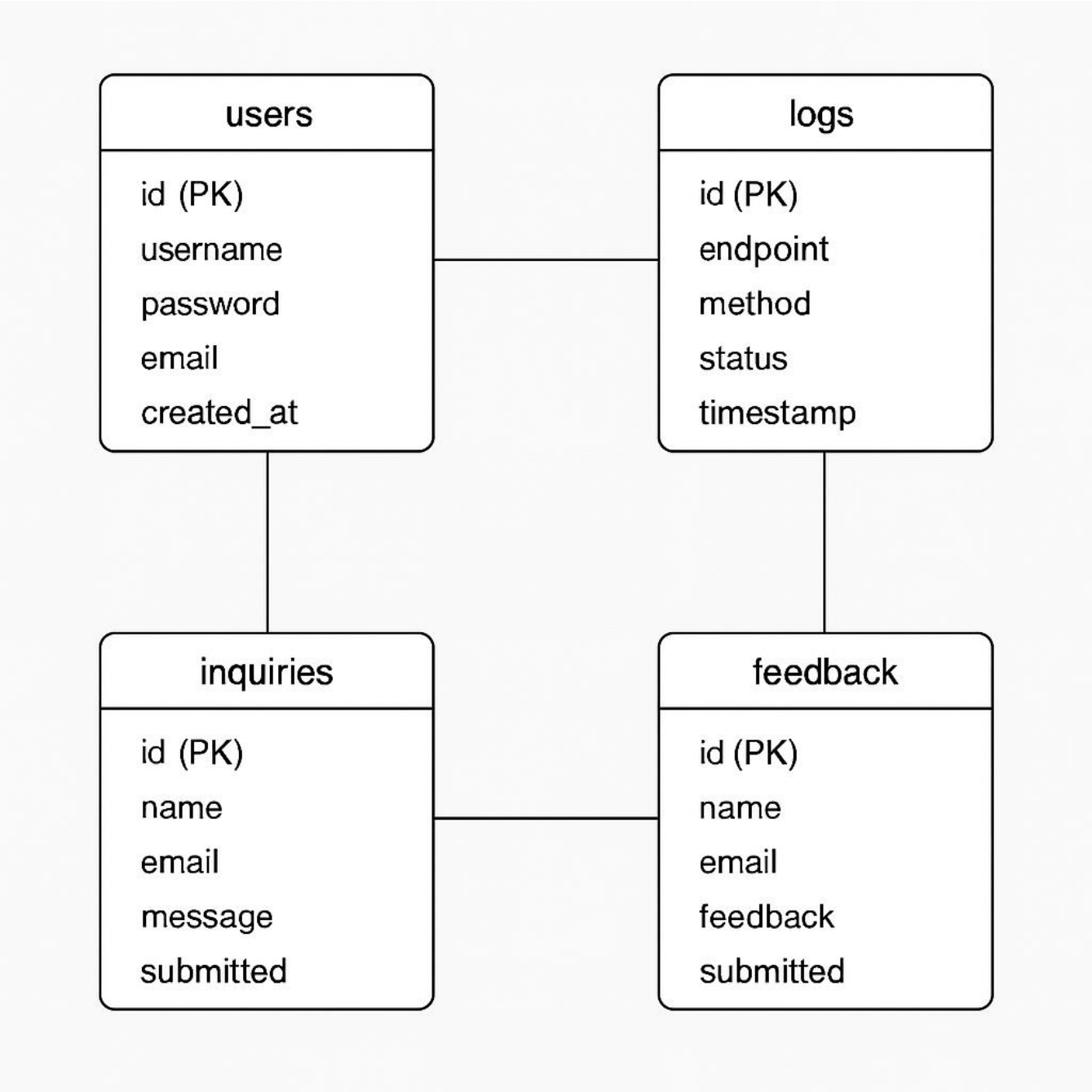
13. Database Schema

Include ER diagram and explanation of tables:

- `users` – Admin credentials
- `logs` – CORS and request logs
- `inquiries` – Inquiry form data
- `feedback` – Feedback form responses

ER Diagram:

ER Diagram (Text-Based Representation)



Explanation of Tables

1. `users` Table – Admin Credentials

Column	Type	Description
`id`	Integer (PK)	Unique ID for each admin user
`username`	String	Admin username
`password`	String (hashed)	Admin password (hashed)
`email`	String	Admin email

Column	Type	Description
`created_at`	DateTime	Timestamp when the user was created

Used for authentication and admin login functionality.

2. `logs` Table – CORS and API Request Logs

Column	Type	Description
`id`	Integer (PK)	Unique log entry ID
`endpoint`	String	API endpoint hit (`/feedback`, `/secure-data`, etc.)
`method`	String	HTTP method used (GET, POST, OPTIONS, etc.)
`status`	Integer	Response status code (e.g., 200, 401, etc.)
`timestamp`	DateTime	Time of the request

Automatically stores CORS-related interactions and general API logs.

3. `inquiries` Table – Inquiry Form Submissions

Column	Type	Description
`id`	Integer (PK)	Unique inquiry ID
`name`	String	Name of person submitting inquiry
`email`	String	Email of the person
`message`	Text	Inquiry message
`submitted`	DateTime	Timestamp of submission

Captures data from the Inquiry form on the frontend.

4. `feedback` Table – Feedback Form Responses

Column	Type	Description
`id`	Integer (PK)	Unique feedback ID
`name`	String	Name of the user giving feedback

Column	Type	Description
`email`	String	Email of the user
`feedback`	Text	Feedback text
`submitted`	DateTime	When the feedback was given

Captures data from the Feedback form.

How These Tables Work Together

- When a **user submits a form** (inquiry or feedback), it's saved into the corresponding table (`inquiries`, `feedback`).
- Every **API request** (public or secure) generates an entry in the `logs` table, including method, endpoint, and status (CORS-relevant).
- **Admins** can log in using credentials in the `users` table and monitor submissions/logs.

14. Attack Demonstration

- Cross-origin request from untrusted domain
- Credential theft using malicious JavaScript
- Insecure admin panel exposure
- Log CORS headers and show improper handling

16. Testing and Result Analysis

Attack Demonstration for CORS Project

1. Cross-Origin Request from Untrusted Domain

Scenario:

Simulate a malicious site (e.g., `http://evil-site.com`) attempting to access your secure API route (e.g., `http://127.0.0.1:5000/secure-data`).

How to Demonstrate:

- Create a simple HTML page named `evil.html` hosted on a different origin (`127.0.0.1:5501` or use [Live Server]).

```
<!-- evil.html -->
<script>
  fetch("http://127.0.0.1:5000/secure-data", {
    method: "GET",
    headers: {
      Authorization: "Bearer demo-token"
    }
  })
  .then(res => res.json())
  .then(data => console.log("Hacked Data:", data))
  .catch(err => console.error("Blocked by CORS:", err));
</script>
```

- The browser **should block this** unless your Flask backend is **improperly configured to allow any origin**.

CORS prevents this if your backend only allows trusted origins using:

```
CORS(app, origins=["http://127.0.0.1:5500"])
```

2. Credential Theft using Malicious JavaScript

Scenario:

A malicious website tricks the user into visiting it. It runs JavaScript trying to steal session/token or POST sensitive data like login credentials.

How to Demonstrate:

In `evil.html`:

```
<form id="stealForm">
  <input type="hidden" name="email" value="victim@example.com">
  <input type="hidden" name="message" value="Stolen via script">
</form>

<script>
  fetch("http://127.0.0.1:5000/inquiry", {
    method: "POST",
```

```
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      name: "Attacker",
      email: "stolen@example.com",
      message: "Sending data cross-origin"
    })
  }).then(res => res.text())
    .then(data => console.log("Form sent", data))
    .catch(e => console.error("CORS blocked!", e));
</script>
```

If CORS is disabled or misconfigured, this cross-origin form **would work** and compromise security.

3. Insecure Admin Panel Exposure

Scenario:

Your `/admin` panel is exposed without CORS or auth protection.

How to Demonstrate:

- Make a route like `/admin/data` return sensitive info.
- Access it from an external `evil.html` using fetch.
- If it's unprotected and CORS allows `*`, attacker can see admin data.

Fix:

- Protect `/admin` with authentication.
- Restrict CORS:

```
CORS(app, resources={
  r"/admin/*": {"origins": "http://127.0.0.1:5500"}
})
```

---

4. Log CORS Headers and Show Improper Handling

How to Demonstrate:

- Log incoming `Origin`, `Access-Control-Allow-Origin`, `Authorization` in your `logs` table.
- In Flask, add logging middleware:

```
@app.after_request
def log_cors_headers(response):
    origin = request.headers.get("Origin", "No Origin")
    log = Logs(endpoint=request.path, status=response.status_code, origin=origin)
    db.session.add(log)
    db.session.commit()
    return response
```

Show in frontend:

Display logs on `logs.html` via JavaScript fetch request to `/logs`.

Tips for Your Demo Presentation:

- Use Chrome DevTools → Network tab to **show blocked CORS** and missing `Access-Control-Allow-Origin` headers.
- Switch between proper and insecure `CORS()` configurations in Flask and show **effects in real-time**.
- Compare working vs blocked requests for `POST` and `GET`.

17. Future Enhancements

- AI-based CORS log anomaly detection
- Blockchain-based origin validation
- Token rotation and zero-trust policies
- Integration with cloud access gateways

19. Conclusion

Summarize:

- Need for CORS in modern web development
- Risk of misconfigurations
- Role of monitoring and admin tools
- Importance of best practices in API development

20. References

- Flask-CORS Docs: https://flask-cors.readthedocs.io
- MDN Web Docs – CORS: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
- OWASP API Security Top 10
- Real-world vulnerability reports